

SPM UNIT-2

1. Explain how Gantt charts, PERT/CPM, SWOT analysis, and work breakdown structures (WBS) assist in effective project planning.

A. 1. Gantt Charts

- **Function:** Visual timeline showing tasks, durations, and dependencies.
- **Use in Planning:**
 - Maps out start and end dates for each activity.
 - Highlights task overlaps and sequencing.
 - Tracks progress against planned milestones.
- **Impact:** Enhances scheduling visibility and coordination among teams.

2. PERT (Program Evaluation Review Technique) / CPM (Critical Path Method)

- **Function:** Network-based scheduling tools that model task dependencies and durations.
- **Use in Planning:**
 - **PERT** handles uncertainty using optimistic, pessimistic, and most likely time estimates.
 - **CPM** identifies the longest path (critical path) that determines project duration.
 - Both help allocate resources and buffer time effectively.
- **Impact:** Improves time estimation, identifies bottlenecks, and supports risk-aware scheduling.

3. SWOT Analysis

- **Function:** Strategic tool to assess internal and external project factors.
- **Use in Planning:**
 - Identifies **Strengths** (e.g., skilled team), **Weaknesses** (e.g., limited budget), **Opportunities** (e.g., market demand), and **Threats** (e.g., regulatory changes).
 - Guides scope definition and risk mitigation strategies.
- **Impact:** Aligns project goals with organizational strategy and prepares for external challenges.

4. Work Breakdown Structure (WBS)

- **Function:** Hierarchical decomposition of project deliverables into manageable tasks.
- **Use in Planning:**
 - Defines scope clearly by breaking down work into components.
 - Facilitates task assignment, cost estimation, and progress tracking.
 - Serves as a foundation for scheduling and budgeting.
- **Impact:** Improves clarity, accountability, and control over project execution.

2. Explain how funds flow analysis helps in identifying the financial health of a project, budgeting, and financial decision-making.

A. 1. Understanding Financial Health

- **Funds flow analysis** examines changes in working capital between two balance sheet dates.
- It highlights whether funds are being generated internally (e.g., profits, depreciation) or externally (e.g., loans, equity).
- **Key insight:** A project with consistent internal fund generation is financially sound; reliance on external sources may indicate stress.
- It also identifies **non-operational uses of funds** (e.g., asset purchases) that may strain liquidity.

2. Supporting Budgeting

- By analyzing **sources and applications of funds**, managers can:
 - Allocate budgets more accurately across departments or phases.
 - Forecast future funding needs based on past patterns.
 - Detect inefficiencies or overspending in specific areas.
- It complements cash flow analysis by focusing on **long-term financial movements**, not just immediate liquidity.

3. Enabling Financial Decision-Making

- Funds flow statements guide decisions on:
 - **Capital investment:** Whether surplus funds can be reinvested or need to be conserved.
 - **Debt management:** Whether borrowing is sustainable or needs restructuring.
 - **Working capital optimization:** Identifying areas where funds are tied up unnecessarily.
- It also aids in evaluating the **impact of strategic choices** (e.g., expansion, divestment) on fund movement.

3. Explain cost-benefit analysis and its role in software project selection and prioritization.

A. **Cost-Benefit Analysis (CBA)** is a financial evaluation technique used to assess whether the benefits of a software project justify its costs. It plays a central role in project selection and prioritization by enabling decision-makers to compare alternatives based on economic value.

1. Core Concept of Cost-Benefit Analysis

- **Objective:** Quantify and compare all expected costs and benefits of a project.
- **Formulaic Basis:**
 - **Total Benefits – Total Costs = Net Benefit**
 - Alternatively, compute **Benefit-Cost Ratio (BCR) = Total Benefits / Total Costs**
- **Interpretation:**
 - If **Net Benefit > 0** or **BCR > 1**, the project is financially viable.
 - Among multiple projects, the one with the highest net benefit or BCR is prioritized.

2. Types of Costs and Benefits in Software Projects

- **Costs:**

- Development costs (labor, tools, infrastructure)
 - Maintenance and support
 - Training and deployment
- **Benefits:**
 - Increased revenue or market share
 - Cost savings (e.g., automation, efficiency)
 - Strategic gains (e.g., competitive advantage, compliance)

3. Role in Project Selection

- **Screening Tool:** Filters out projects with poor financial justification.
- **Comparison Basis:** Enables ranking of multiple proposals based on economic return.
- **Risk Awareness:** Highlights projects with uncertain or delayed payoffs.

4. Role in Prioritization

- **Resource Allocation:** Directs funds and talent to high-return initiatives.
- **Portfolio Optimization:** Balances short-term gains with long-term strategic value.
- **Stakeholder Communication:** Provides transparent rationale for investment decisions.

5. Limitations and Mitigation

- **Intangible Benefits:** User satisfaction, brand value may be hard to quantify.
- **Assumption Sensitivity:** Estimates may be biased or uncertain.
- **Mitigation:** Use sensitivity analysis, scenario modeling, and combine with scoring models or strategic filters.

4. What is risk evaluation in software project management?

A. 1. Purpose of Risk Evaluation

- After risks are identified (e.g., scope creep, technology failure, resource shortages), evaluation helps prioritize them.
- It answers: *Which risks matter most?* and *What should we do about them?*

2. Key Components of Risk Evaluation

- **Probability (Likelihood):** How likely is the risk to occur? (e.g., low, medium, high)
- **Impact (Severity):** What is the potential effect on cost, schedule, quality, or scope?
- **Risk Exposure (RE):** Often calculated as

$$RE = \text{Probability} \times \text{Impact}$$
- **Risk Ranking:** Risks are ranked based on RE to focus attention on the most critical ones.

3. Qualitative vs. Quantitative Evaluation

- **Qualitative:**
 - Uses categories (e.g., high/medium/low) for probability and impact.
 - Useful in early stages or when data is limited.
- **Quantitative:**
 - Assigns numerical values to probability and impact.

- Enables simulation (e.g., Monte Carlo), decision trees, or expected monetary value (EMV) analysis.

4. Outcome of Risk Evaluation

- **Risk Prioritization:** Focuses mitigation efforts on high-exposure risks.
- **Response Planning:** Guides selection of strategies—avoid, transfer, mitigate, or accept.
- **Contingency Allocation:** Helps allocate time and budget buffers.

5. How does portfolio management contribute to strategic alignment of software projects?

A. 1. Strategic Filtering and Selection

- Portfolio management evaluates proposed projects based on their contribution to strategic objectives (e.g., digital transformation, market expansion, cost reduction).
- Projects that align with key business drivers are prioritized; misaligned or redundant initiatives are deferred or rejected.
- This ensures that **resources are not wasted on low-value or off-strategy efforts**.

2. Balanced Resource Allocation

- It provides a **portfolio-wide view of resource demand**, helping avoid overcommitment or underutilization.
- By balancing across project types (e.g., innovation vs. maintenance), it ensures a **diversified investment mix** that supports both short-term performance and long-term growth.

3. Risk and Value Optimization

- Portfolio managers assess **risk-return profiles** of projects collectively, not just individually.
- High-risk projects may be balanced with low-risk, high-certainty ones to maintain overall portfolio stability.
- This supports **informed trade-offs** between innovation, compliance, and operational efficiency.

4. Performance Monitoring and Realignment

- Through **continuous tracking of KPIs**, portfolio management identifies underperforming or misaligned projects early.
- It enables **dynamic reallocation**—shifting funds, scope, or timelines to better support strategic shifts or respond to market changes.

5. Stakeholder Communication and Governance

- Provides a **transparent framework** for executives to understand how each project contributes to strategic goals.
- Facilitates **evidence-based decision-making** through dashboards, scorecards, and scenario analysis.

6. Define benefit management and discuss its importance in software project success.

A. **Benefit management** is the structured process of identifying, planning, measuring, and sustaining the value a software project is expected to deliver. It ensures that the project's outcomes align with strategic goals and that the intended benefits are realized and sustained after implementation.

1. Definition of Benefit Management

- It involves four key stages:
 1. **Identification:** Define expected benefits (e.g., cost savings, user satisfaction, market growth).
 2. **Planning:** Establish how and when benefits will be delivered, and who is accountable.
 3. **Realization:** Monitor and measure benefits during and after project execution.
 4. **Sustainment:** Ensure benefits continue post-deployment through operational support and change management.

2. Importance in Software Project Success

a. Strategic Alignment

- Ensures that the project contributes to organizational goals (e.g., digital transformation, customer retention).
- Prevents investment in technically sound but strategically irrelevant projects.

b. Value-Driven Decision-Making

- Shifts focus from just delivering features to delivering measurable outcomes.
- Helps prioritize features or modules based on their benefit contribution.

c. Stakeholder Engagement

- Clarifies what success looks like for different stakeholders.
- Facilitates buy-in by linking project outputs to tangible business value.

d. Performance Measurement

- Provides metrics to evaluate project success beyond time, cost, and scope (e.g., ROI, NPS, productivity gains).
- Enables course correction if benefits are not materializing as planned.

e. Post-Implementation Accountability

- Encourages ownership beyond project closure.
- Supports continuous improvement and benefit tracking during operations.

3. Example in Practice

Suppose a software project aims to automate invoice processing:

- **Identified Benefit:** Reduce processing time by 50%.
- **Planned Realization:** Within 3 months of deployment.
- **Measurement:** Compare average processing time before and after.
- **Sustainment:** Train staff and monitor KPIs quarterly.

7. Explain the techniques used in gathering requirements during the initial stages of a software project.

A. 1. Interviews

- **Purpose:** Direct conversations with stakeholders to understand goals, constraints, and expectations.

- **Types:** Structured (predefined questions), unstructured (open-ended), or semi-structured.
- **Strengths:** Deep insights, flexible follow-up.
- **Limitations:** Time-consuming; may miss broader perspectives.

2. Questionnaires and Surveys

- **Purpose:** Collect standardized input from a large group.
- **Strengths:** Efficient for distributed teams or large user bases.
- **Limitations:** Limited depth; depends on question clarity.

3. Workshops and Focus Groups

- **Purpose:** Collaborative sessions to brainstorm, prioritize features, and resolve conflicts.
- **Strengths:** Builds consensus; exposes diverse viewpoints.
- **Limitations:** Requires skilled facilitation; may be dominated by vocal participants.

4. Observation (Job Shadowing)

- **Purpose:** Watch users perform tasks to uncover implicit requirements.
- **Strengths:** Reveals real-world workflows and pain points.
- **Limitations:** May not capture edge cases or future needs.

5. Document Analysis

- **Purpose:** Review existing manuals, reports, or legacy systems.
- **Strengths:** Identifies baseline functionality and compliance needs.
- **Limitations:** May reflect outdated or irrelevant practices.

6. Prototyping

- **Purpose:** Build mock-ups or interactive models to elicit feedback.
- **Strengths:** Clarifies UI/UX expectations; reduces misinterpretation.
- **Limitations:** May lead to premature design fixation.

7. Brainstorming and Mind Mapping

- **Purpose:** Generate ideas and explore requirement domains.
- **Strengths:** Encourages creativity and broad exploration.
- **Limitations:** Needs refinement and validation.

8. Describe the process of identifying project scope and preparing a scope statement.

A. To identify project scope and prepare a scope statement, a project manager follows a structured process that defines what the project will deliver, what it will not, and how boundaries will be managed. This ensures clarity, alignment, and control throughout the project lifecycle.

1. Requirement Collection

- Gather stakeholder needs using interviews, surveys, workshops, and document analysis.
- Translate business goals into functional and non-functional requirements.
- Output: Initial list of deliverables and constraints.

2. Define Project Objectives

- Clarify what the project aims to achieve (e.g., develop a mobile app for online

banking).

- Align objectives with organizational strategy and stakeholder expectations.

3. Identify Deliverables

- List tangible outputs (e.g., software modules, documentation, training materials).
- Specify acceptance criteria for each deliverable.

4. Determine Boundaries and Exclusions

- Explicitly state what is **not** included (e.g., no support for legacy browsers).
- Prevent scope creep by setting clear limits.

5. Develop Work Breakdown Structure (WBS)

- Decompose deliverables into manageable tasks and sub-tasks.
- Use WBS to support scheduling, budgeting, and resource planning.

6. Assess Constraints and Assumptions

- Document known limitations (e.g., budget cap, fixed deadline).
- Note assumptions (e.g., availability of skilled developers, stable API access).

7. Draft the Scope Statement

- Include:
 - Project justification
 - Objectives and deliverables
 - Boundaries and exclusions
 - Constraints and assumptions
 - Acceptance criteria
- Ensure it is reviewed and approved by key stakeholders.

9. Explain the step-wise project planning process proposed by software engineering standards.

A. 1. Project Initiation and Scope Definition

- **Objective:** Establish the purpose, goals, and boundaries of the project.
- **Activities:**
 - Define project objectives and success criteria.
 - Identify stakeholders and their expectations.
 - Draft a high-level scope statement.

2. Feasibility and Resource Assessment

- **Objective:** Evaluate technical, economic, and operational feasibility.
- **Activities:**
 - Conduct cost-benefit analysis.
 - Assess resource availability (personnel, tools, infrastructure).
 - Identify constraints and assumptions.

3. Work Breakdown Structure (WBS) Development

- **Objective:** Decompose the project into manageable components.
- **Activities:**
 - Break down deliverables into tasks and subtasks.
 - Assign responsibilities and define task dependencies.
 - Use WBS as a foundation for scheduling and budgeting.

4. Schedule and Milestone Planning

- **Objective:** Develop a realistic timeline for task execution.
- **Activities:**
 - Estimate task durations.
 - Sequence tasks using Gantt charts or PERT/CPM.
 - Define key milestones and deadlines.

5. Resource and Cost Planning

- **Objective:** Allocate resources and estimate project costs.
- **Activities:**
 - Assign personnel, tools, and facilities to tasks.
 - Estimate effort and cost per task.
 - Prepare a project budget and funding plan.

6. Risk Management Planning

- **Objective:** Identify and prepare for potential project risks.
- **Activities:**
 - Perform risk identification and evaluation.
 - Develop mitigation and contingency plans.
 - Assign risk ownership and monitoring procedures.

7. Quality Assurance Planning

- **Objective:** Define how software quality will be ensured.
- **Activities:**
 - Set quality objectives and standards.
 - Plan reviews, audits, and testing strategies.
 - Define defect tracking and resolution processes.

8. Communication and Stakeholder Management

- **Objective:** Ensure effective information flow.
- **Activities:**
 - Define communication channels and reporting formats.
 - Schedule status meetings and progress updates.
 - Manage stakeholder expectations and feedback.

9. Configuration and Change Management Planning

- **Objective:** Control changes to project artifacts.
- **Activities:**
 - Define version control and change request procedures.
 - Establish baselines and approval workflows.
 - Integrate with risk and quality plans.

10. Plan Consolidation and Approval

- **Objective:** Finalize and baseline the project plan.
- **Activities:**
 - Integrate all planning components into a master plan.
 - Review with stakeholders and sponsors.
 - Obtain formal approval and initiate execution.

10. Discuss various software effort estimation techniques: expert judgment,

analogy-based estimation, parametric models like COCOMO and machine learning-based estimation methods.

A. 1. Expert Judgment

- **Definition:** Relies on the intuition and experience of domain experts to estimate effort.
- **Process:** Experts review project scope, complexity, and constraints, then provide estimates individually or through consensus (e.g., Delphi method).
- **Strengths:**
 - Fast and flexible.
 - Useful when historical data is unavailable.
- **Limitations:**
 - Subjective and prone to bias.
 - Accuracy depends on expert experience and domain familiarity.

2. Analogy-Based Estimation

- **Definition:** Estimates effort by comparing the current project to past, similar projects.
- **Process:**
 - Identify analogous projects.
 - Adjust for differences in size, complexity, or technology.
- **Strengths:**
 - Leverages real-world data.
 - Intuitive and explainable.
- **Limitations:**
 - Requires a well-maintained project history database.
 - Difficult if no close analog exists.

3. Parametric Models (e.g., COCOMO)

- **Definition:** Use mathematical formulas based on project attributes (e.g., lines of code, function points) to estimate effort.
- **COCOMO (Constructive Cost Model):**
 - **Basic COCOMO:** $\text{Effort} = a \times (\text{KLOC})^b$
 - **Intermediate/COCOMO II:** Includes cost drivers like reliability, team experience, and tool support.
- **Strengths:**
 - Structured and repeatable.
 - Scalable for different project sizes.
- **Limitations:**
 - Requires accurate input metrics.
 - May not adapt well to modern agile or component-based development.

4. Machine Learning-Based Estimation

- **Definition:** Applies algorithms (e.g., regression, decision trees, neural networks) to learn patterns from historical project data.
- **Process:**
 - Train models on features like team size, complexity, domain, and past effort.

- Predict effort for new projects based on learned relationships.
- **Strengths:**
 - Can capture nonlinear and complex relationships.
 - Improves over time with more data.
- **Limitations:**
 - Requires large, clean datasets.
 - Interpretability can be low (especially with deep learning).

11. Discuss the impact of schedule compression on software project cost, quality, and risk.

A. **Schedule compression** refers to techniques used to shorten the project timeline without reducing scope. Common methods include **crashing** (adding resources) and **fast-tracking** (overlapping tasks). While it may help meet deadlines, it has significant implications for **cost, quality, and risk**.

1. Impact on Cost

- **Crashing** increases cost due to:
 - Overtime payments
 - Hiring additional staff or consultants
 - Expedited procurement or tool acquisition
- **Fast-tracking** may reduce direct costs but can lead to:
 - Rework due to premature task execution
 - Hidden costs from coordination failures
- **Net Effect:** Cost often increases unless compression is carefully planned and justified by business value.

2. Impact on Quality

- **Reduced testing time** leads to:
 - Missed defects
 - Incomplete validation of edge cases
- **Stress on teams** causes:
 - Burnout and reduced attention to detail
 - Lower code maintainability and documentation quality
- **Net Effect:** Quality degradation is likely unless mitigated by automation, modular design, or parallel QA.

3. Impact on Risk

- **Technical risk** increases due to:
 - Overlapping tasks without full dependency resolution
 - Inadequate integration and regression testing
- **Management risk** rises from:
 - Resource conflicts
 - Communication breakdowns under pressure
- **Net Effect:** Risk exposure grows, especially in complex or interdependent systems.

12. Explain the process and challenges involved in software cost estimation: direct

vs. indirect costs, fixed vs. variable costs, and internal vs. external costs.

A. 1. Process of Software Cost Estimation

a. Requirement Analysis

- Understand the scope, features, and constraints of the project.
- Identify functional and non-functional requirements.

b. Work Breakdown Structure (WBS)

- Decompose the project into smaller, manageable tasks.
- Assign effort estimates to each task.

c. Choose Estimation Techniques

- Use one or more of the following:
 - Expert judgment
 - Analogy-based estimation
 - Parametric models (e.g., COCOMO)
 - Machine learning or historical data analysis

d. Identify Cost Categories

- Classify costs into direct/indirect, fixed/variable, internal/external.

e. Aggregate and Adjust

- Sum all estimated costs.
- Add contingency buffers for risk and uncertainty.

f. Review and Validate

- Cross-check with historical data or expert feedback.
- Revise based on stakeholder input or scope changes.

2. Cost Classifications and Their Role

Cost Type	Description	Examples	Estimation Challenge
Direct Costs	Attributable to specific project tasks	Developer salaries, software licenses	Easy to identify but may vary with scope
Indirect Costs	Shared across multiple projects or operations	Admin overhead, utilities	Hard to allocate proportionally
Fixed Costs	Do not vary with project size or duration	Hardware purchase, office rent	Easier to estimate but may be underestimated
Variable Costs	Change with project scale or duration	Contract labor, cloud usage	Sensitive to scope creep or delays
Internal Costs	Incurred using in-house resources	Salaries, internal tools	Often underestimated due to sunk cost bias
External Costs	Paid to third parties	Consultants, outsourced testing	Prone to market rate fluctuations

3. Challenges in Software Cost Estimation

- **Unclear or evolving requirements:** Leads to inaccurate scope and effort estimates.
- **Lack of historical data:** Makes analogy or parametric models less reliable.
- **Human bias:** Optimism bias or anchoring can distort expert judgment.

- **Technology uncertainty:** New tools or platforms introduce unknowns.
- **Integration and rework:** Often underestimated in early stages.
- **Changing team dynamics:** Productivity varies with experience, turnover, and collaboration.